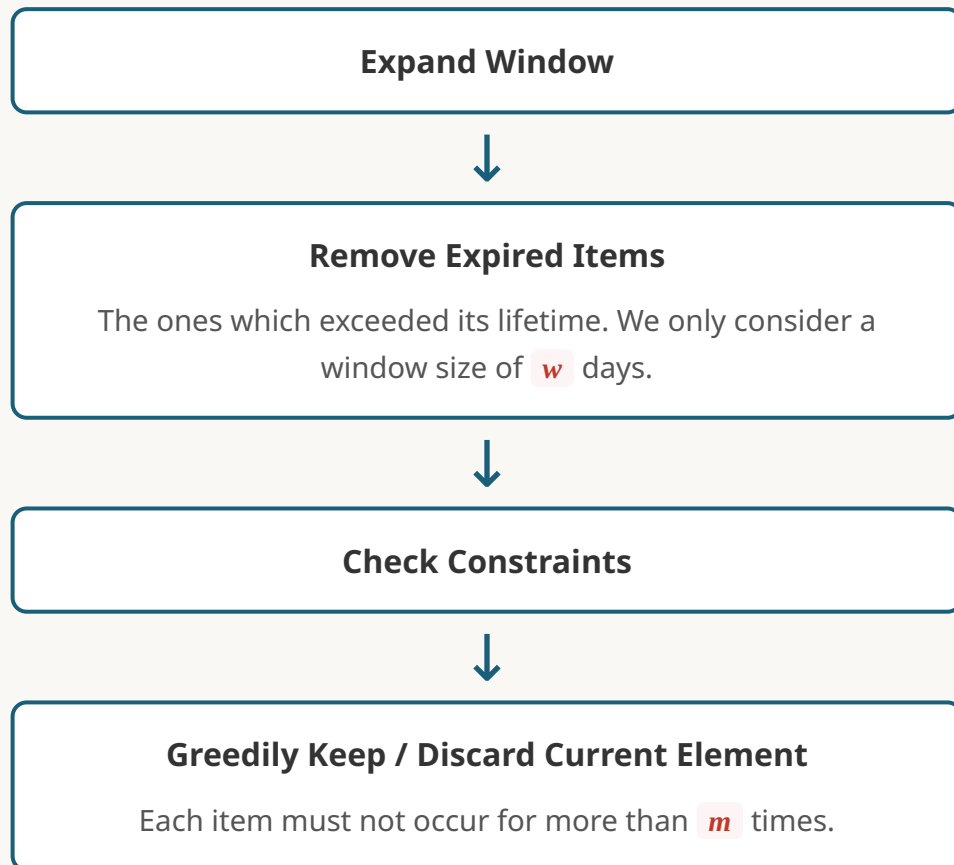


LC 3679: Minimum Discards to Balance Inventory

1. Core Strategy / Algorithm Flow



2. Key Observations

Observation 1: Outgoing Elements

Remove every item whose day is $< j - w + 1$. This represents the outgoing element from our sliding window context.

Observation 2: The Greedy Choice

If an item has reached its max pickup count, then simply discard the item on the current day. This is a greedy choice because the oldest item is anyways going to leave soon, so discarding the newly arrived one (or keeping the older valid ones) optimizes our constraints moving forward.

3. Data Structures & Complexity

Data Structure Used

We utilize an `unordered_map` mapping the items to a `queue` of their arrival days.

This allows us to track exactly when each specific item was picked up and efficiently discard expired occurrences from the front of the queue.

Complexity Analysis

Time Complexity (TC): $O(n)$

Each element is pushed and popped from the queue at most once.

Space Complexity (SC): $O(n)$

In the worst-case scenario, to store the queues inside the hash map.

4. Implementation (C++)

```
class Solution {
public:
    int minArrivalsToDiscard(vector<int>& arrivals, int w, int m) {
        int n = arrivals.size();
        unordered_map<int, queue<int>> mp;
        int discards = 0;

        for(int j = 1; j <= n; j++){ // days are 1 indexed

            int item = arrivals[j-1];

            while(!mp[item].empty() && mp[item].front() < j - w + 1){
                mp[item].pop();
            }

            if(mp[item].size() == m){
                discards++;
            }
            else{
                mp[item].push(j);
            }
        }

        return discards;
    }
};
```